



DESENVOLVIMENTO WEB

Revisão para Prova

Módulo 2 · Desenvolvimento Web

Banco de Dados

MVC & Testes

RF / RNF / RTM

Diagramas UML

Padrões de Projeto



Formatos ENADE: Somatória e Asserção

Leia cada afirmativa isoladamente antes de olhar as alternativas. Em asserção-razão, avalie verdade e relação de causa.

SOMATÓRIA

Cada afirmativa tem peso em potência de 2. Some apenas as verdadeiras.

- I. GET não deve alterar estado. (V) = 1
- II. JOIN exige sempre PK. (F) = 2
- III. beforeEach roda antes de cada it. (V) = 4
- IV. RNF pode ser vago. (F) = 8

Resposta: $I + III = 1 + 4 = 5$

ASSERÇÃO-RAZÃO

Pode ser V-V explicativa, V-V sem explicação, V-F, F-V ou F-F.

Teste unitário de Service deve mockar Repository
PORQUE
o objetivo é validar a regra de negócio isolada.

Ambas verdadeiras, e a razão explica.



Formatos ENADE: Composta e Dissertativa

Nas compostas, elimine alternativas que trazem uma sentença claramente falsa. Na dissertativa, responda com conceito, motivo e exemplo.

MÚLTIPLA ESCOLHA COMPOSTA

Identifique quais sentenças estão corretas e escolha a alternativa compatível.

- I. Model conhece req/res do Express. (F)
- II. FK garante integridade referencial. (V)
- III. Sequência mostra interação no tempo. (V)
- IV. Classe de domínio = implantação. (F)

Corretas: II e III

DISSERTATIVA

Use frases curtas, exemplo concreto e, se pedirem, código ou diagrama simples.

Explique teste unitário vs integração:

1. Defina cada tipo.
2. Diga o que mocka.
3. Dê um exemplo de camada.



MER: Modelo Conceitual

IDEIA CENTRAL

O MER descreve quais entidades existem no domínio, seus atributos e como elas se relacionam. Ele ainda não depende de SQL, ORM ou SGBD.

COMPONENTES QUE PRECISAM APARECER

- Entidades: objetos com existência própria, como Cliente, Pedido e Produto.
- Atributos: propriedades relevantes, como nome, email, data e preço.
- Relacionamentos: verbos do domínio, como realiza, contém e pertence.
- Cardinalidades: 1:1, 1:N e N:M, lidas nos dois sentidos.

REGRA IMPORTANTE

Relacionamento N:M no conceitual vira tabela associativa no físico. Exemplo: Aluno cursa Disciplina vira Matricula(aluno_id, disciplina_id).



MER: Exemplo que Atende aos Requisitos



POR QUE ESTÁ CORRETO

- O domínio está claro: biblioteca com usuários, livros, autores e empréstimos.
- Há 1:N entre Usuário e Empréstimo; 1:N entre Livro e Empréstimo.



DER: Representação Gráfica do Banco

DER VS MER

O MER é o modelo conceitual. O DER é o desenho desse modelo, geralmente com tabelas, chaves e cardinalidades mais próximas do banco.

NOTAÇÃO ÚTIL PARA PROVA

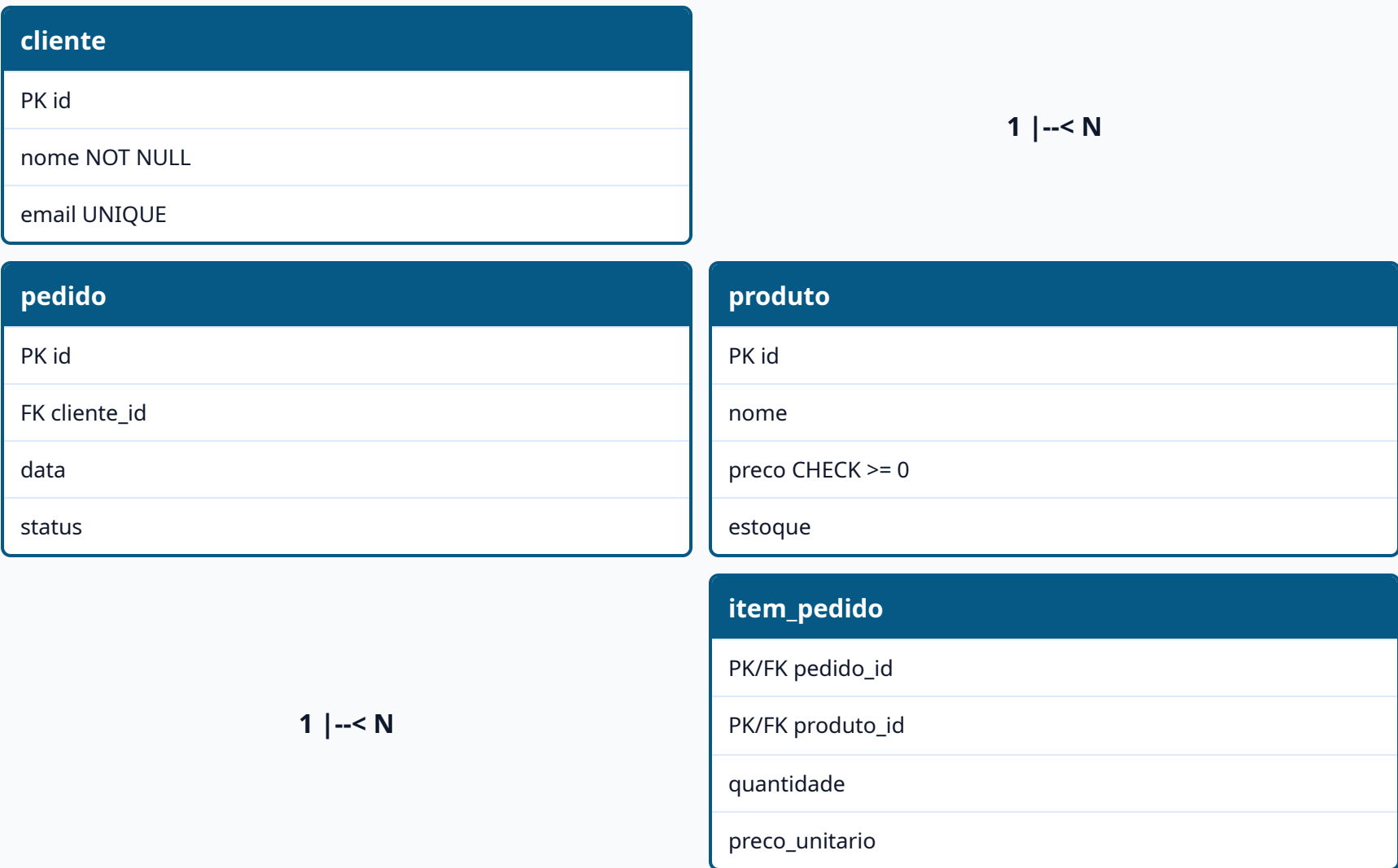
- PK identifica unicamente uma linha.
- FK aponta para a PK de outra tabela.
- 1:N significa que uma instância de A pode se relacionar com várias de B.
- Entidade fraca depende de uma entidade forte, como ItemPedido depende de Pedido.

LEITURA DE CROW'S FOOT

|--< indica um para muitos. O--< indica zero ou muitos. >--< representa muitos para muitos e precisa ser resolvido com tabela associativa.



DER: Exemplo Físico Bem Formado



POR QUE ESTÁ CORRETO

- Pedido tem FK cliente_id, então cada pedido pertence a um cliente.
- ItemPedido usa PK composta e resolve a relação entre Pedido e Produto.



Modelo Físico e Migrations

TRÊS NÍVEIS

- Conceitual: domínio e relações, independente de tecnologia.
- Lógico: tabelas, colunas, chaves e tipos genéricos.
- Físico: SQL executável no PostgreSQL, SQLite, MySQL etc.

CONSTRAINTS ESSENCIAIS

- PRIMARY KEY: identifica a linha.
- FOREIGN KEY: garante referência válida.
- NOT NULL, UNIQUE, DEFAULT e CHECK: restringem valores aceitos.

EXEMPLO CURTO

```
CREATE TABLE cliente (  
  id INTEGER PRIMARY KEY,  
  email TEXT NOT NULL UNIQUE  
);  
  
CREATE TABLE pedido (  
  id INTEGER PRIMARY KEY,  
  cliente_id INTEGER NOT NULL,  
  total NUMERIC CHECK (total >= 0),  
  FOREIGN KEY (cliente_id) REFERENCES cliente(id)  
);
```




BANCO DE DADOS

SQL: CRUD e JOINS

CRUD

- CREATE: INSERT INTO tabela (...) VALUES (...).
- READ: SELECT ... FROM ... WHERE ... ORDER BY ...
- UPDATE: UPDATE tabela SET campo = valor WHERE id = ?.
- DELETE: DELETE FROM tabela WHERE id = ?.

JOINS

- INNER JOIN: só linhas com par dos dois lados.
- LEFT JOIN: todos da esquerda, NULL quando não há par à direita.
- FULL JOIN: mantém linhas sem par dos dois lados.

EXEMPLO DE PROVA

```
SELECT p.id, c.nome AS cliente, p.total
FROM pedido p
INNER JOIN cliente c ON c.id = p.cliente_id
WHERE p.status = 'aberto'
ORDER BY p.id DESC;
```



BACK-END

MVC e Camadas: Fluxo Correto

Mapeia URL e verbo HTTP para o controller correto.



Controller

Valida entrada, chama o service e monta a resposta.



Service

Aplica regras de negócio e orquestra operações.



Repository

Executa SQL e traduz dados do banco para o domínio.



Banco

POR QUE ESTÁ CORRETO

- Controller recebe HTTP e delega; não escreve SQL direto.
- Service concentra regra de negócio e orquestração.



BACK-END

Web API e Endpoints REST

VERBOS

- GET lista ou busca; é idempotente e não deve alterar estado.
- POST cria; não é idempotente.
- PUT substitui; PATCH altera parcialmente.
- DELETE remove; deve ser idempotente no resultado.

STATUS CODES

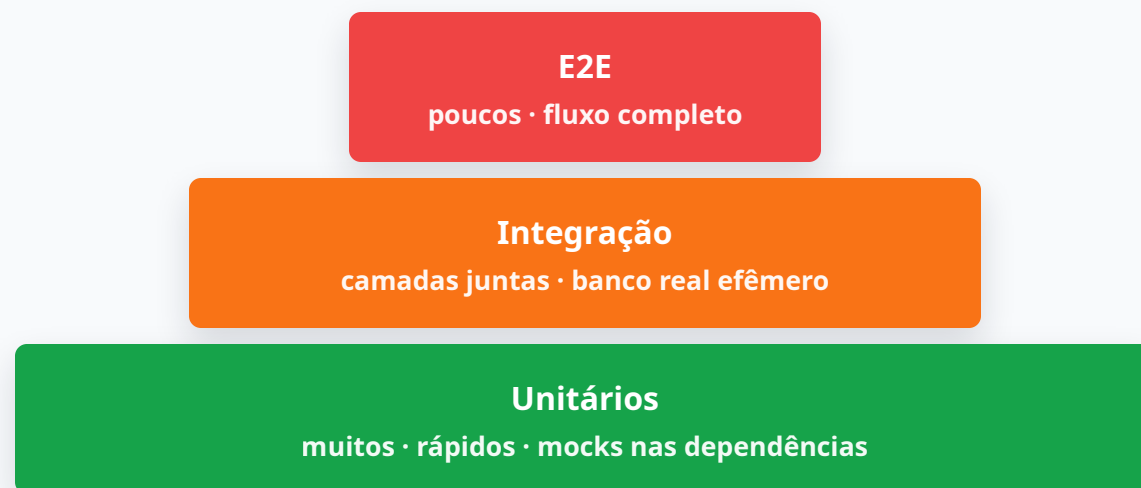
- 200 OK, 201 Created, 204 No Content.
- 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found.
- 409 Conflict para duplicidade; 422 para regra de negócio inválida.

PEGADINHA

401 é falta de autenticação. 403 é usuário autenticado, mas sem permissão para a ação.



Pirâmide de Testes



POR QUE ESTÁ CORRETO

- Unitários são a base: muitos, rápidos e isolados.
- Integração valida a conversa real entre camadas, incluindo banco efêmero.

**TESTES**

Jest: AAA, Mocks e Hooks

AAA

- Arrange: prepara dados, instâncias, mocks e estado inicial.
- Act: executa a função ou requisição uma vez.
- Assert: verifica saída, status e chamadas de mock.

O QUE MOCKAR

- Service: mocka Repository.
- Controller: mocka Service ou usa app com banco em memória.
- Repository: não mocka banco; testa SQL real em banco efêmero.

EXEMPLO UNITÁRIO

```
const repo = { findById: jest.fn().mockResolvedValue(null) };
const service = new UserService(repo);

await expect(service.getById(99))
  .rejects.toThrow('Usuário não encontrado');

expect(repo.findById).toHaveBeenCalledWith(99);
```



RF, RNF e Regra de Negócio

RF

O que o sistema faz. Exemplo: o sistema deve permitir que o usuário redefina senha por e-mail.

RNF

Como o sistema se comporta, com métrica. Exemplo: 95% das requisições devem responder em até 500ms com 200 usuários simultâneos.

RN

Regra do domínio. Exemplo: pedido só pode ser criado se houver estoque suficiente.

ERRO CLÁSSICO

'O sistema deve ser rápido' não é RNF válido, porque não é mensurável.



RTM: Tabela de Rastreabilidade

FUNÇÃO

Conecta necessidade, requisito, regra, implementação e evidência. Ajuda a provar que nada foi esquecido e que nada foi feito sem justificativa.

COLUNAS TÍPICAS

- Persona e necessidade.
- RF, RNF ou regra de negócio associada.
- Endpoint, tela, classe ou arquivo implementado.
- Teste, diagrama ou print que prova a entrega.
- Status: pendente, em andamento ou implementado.

EXEMPLO

```
Persona: gerente de estoque
Necessidade: evitar venda sem estoque
RF: criar pedido
RN: quantidade <= estoque disponível
Implementação: PedidoService.criar()
Evidência: teste criaPedidoSemEstoque.test.js
```



UML: Diagrama de Sequência

QUANDO USAR

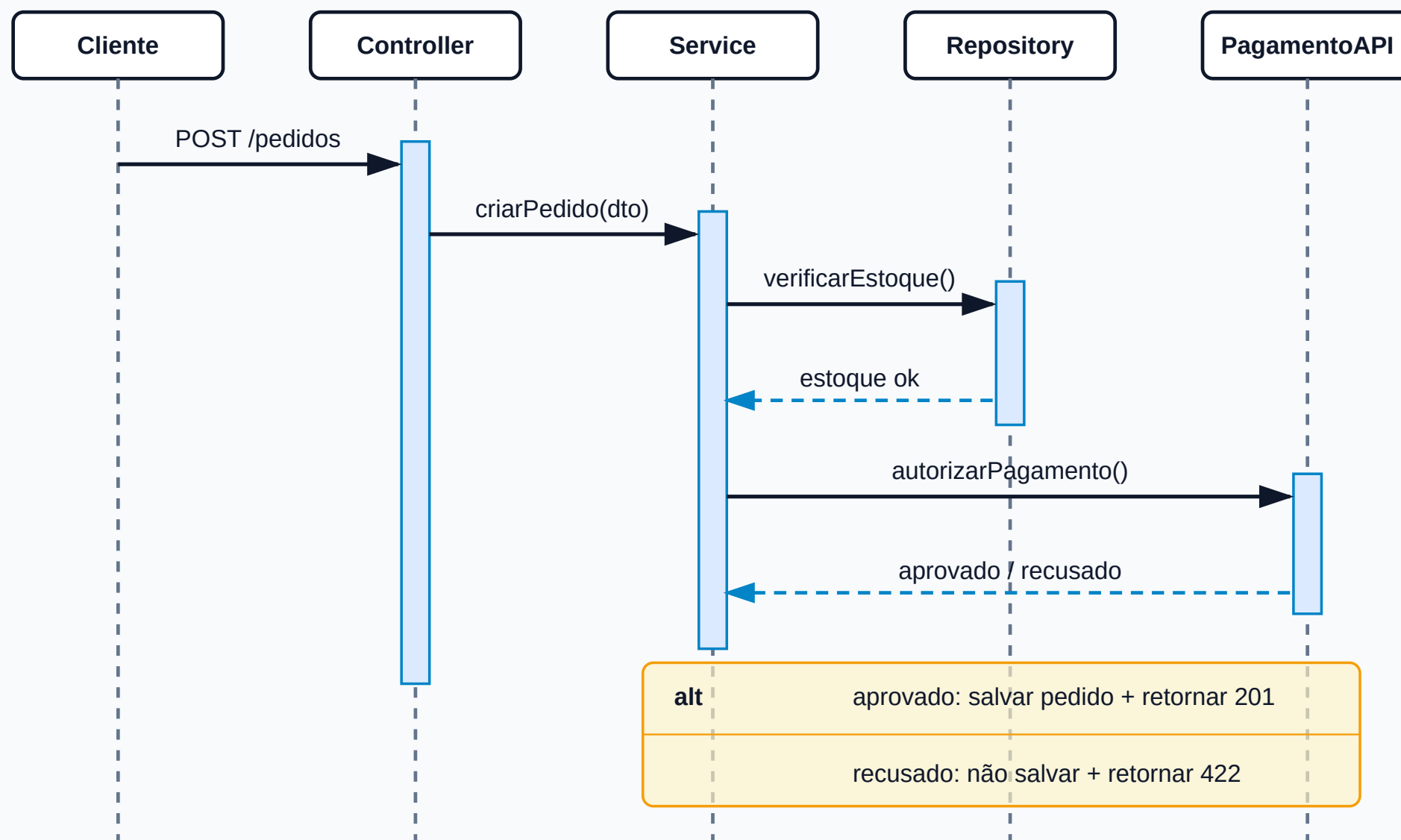
Para documentar um fluxo específico ao longo do tempo: login, checkout, cadastro ou sincronização.

ELEMENTOS OBRIGATÓRIOS

- Participantes no eixo horizontal.
- Tempo descendo no eixo vertical.
- Setas sólidas para chamadas; tracejadas para retornos.
- Ativações mostrando quem está processando.
- Fragmentos alt, opt ou loop quando há condição ou repetição.



Sequência: Criar Pedido com Pagamento



POR QUE ESTÁ CORRETO

- O fluxo tem participantes de fronteira, controle, domínio, persistência e serviço externo.
- As chamadas descem em ordem temporal e os retornos



UML: Classes de Domínio vs Arquitetura

CLASSES DE DOMÍNIO

Mostram conceitos do negócio, atributos e associações. Evitam nomes de camadas e detalhes de framework.

CLASSES ARQUITETURAIS

Mostram classes do código, métodos e dependências entre camadas. É onde entram Controller, Service e Repository.

RELAÇÕES

- Associação: uma classe conhece outra.
- Composição: parte não existe sem o todo.
- Herança: especialização de uma classe base.
- Visibilidade: + público, - privado, # protegido.



Classes de Domínio: Pedido



POR QUE ESTÁ CORRETO

- Não há Controller, Service nem Repository: o foco é o negócio.
- ItemPedido é composição de Pedido: não faz sentido sem o Pedido.



Classes Arquiteturais: Pedido

PedidoController

- +create(req,res)
- +list(req,res)

uses

PedidoService

- +criar(dto)
- +cancelar(id)
- -validarEstoque()

uses

PedidoRepository

- +save(pedido)
- +findById(id)

SQL

Database

- +query(sql, params)
- +transaction(fn)

POR QUE ESTÁ CORRETO

- Mostra dependências reais do código entre camadas.
- Controller depende de Service, e Service depende de Repository.



UML: Diagrama de Implantação

QUANDO USAR

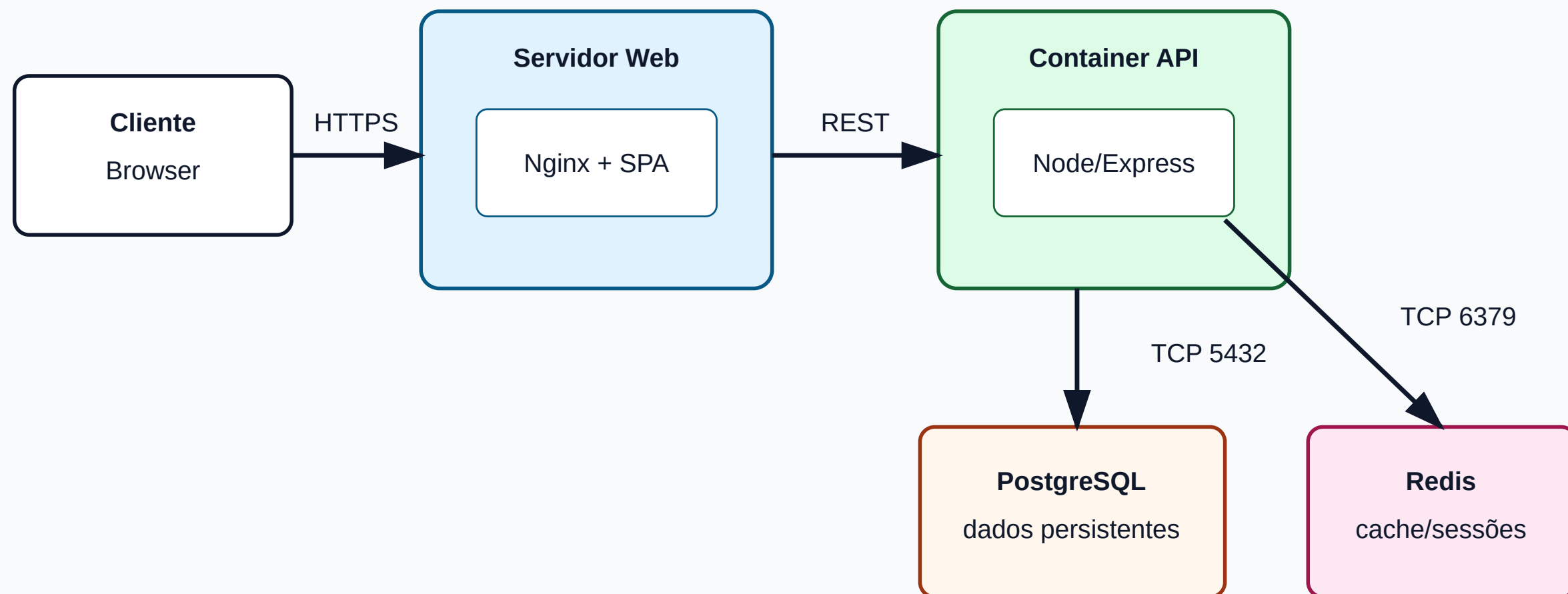
Para responder onde cada parte do sistema roda e como os nós se comunicam em produção.

O QUE PRECISA APARECER

- Nós físicos ou lógicos: navegador, servidor, container, banco.
- Artefatos implantados em cada nó: frontend, API, worker.
- Protocolos ou canais: HTTPS, REST, TCP, fila.
- Dependências externas: gateway de pagamento, e-mail, storage.



Implantação: App Web em Produção



POR QUE ESTÁ CORRETO

- Cada artefato está dentro do nó onde roda.
- As conexões indicam protocolo e direção de comunicação.
- Banco e cache ficam separados da API, como em um



Padrão MVC e Camadas do Projeto

IDEIA CORRETA PARA A PROVA

Neste contexto, o padrão cobrado é a organização arquitetural do sistema: MVC com camadas auxiliares Service e Repository. O foco é saber a responsabilidade de cada camada.

CAMADAS USADAS

- View: interface ou resposta apresentada ao usuário; em API REST, normalmente é JSON.
- Controller: recebe a requisição, valida entrada simples e chama o Service.
- Service: concentra regras de negócio e orquestra o fluxo da operação.
- Repository: isola o acesso ao banco e executa consultas SQL.
- Model: representa os dados e conceitos principais do domínio.

FLUXO ESPERADO

```
View / Cliente HTTP
-> Controller
  -> Service
    -> Repository
      -> Banco de dados
      -> Model / dados do domínio
<- Resposta JSON ou tela
```

ERRO COMUM

Controller não deve conter SQL nem regra de negócio pesada. Repository não deve decidir regra de negócio. Service não deve depender de req/res do Express.



O que Pode Cair por Formato

SOMATÓRIA

- HTTP/status codes
- tipos de teste
- PK/FK/UNIQUE/NOT NULL
- JOINS
- padrões de projeto

ASSERÇÃO-RAZÃO

- Service mocka Repository
- GET é idempotente
- N:M vira associativa
- beforeEach isola testes

COMPOSTA

- MVC correto
- diagramas UML
- RNFs mensuráveis
- migrations

DISSERTATIVA

- SQL com JOIN
- pirâmide de testes
- RF vs RNF
- camadas MVC
- diagrama adequado ao enunciado